

---

---

# Redes Neurais e Aprendizagem Profunda

— Treino de modelos —

---

---

# Sumário

Sistema de AA

Regressão linear

Descida do gradiente

Regressão polinomial

Curvas de aprendizagem

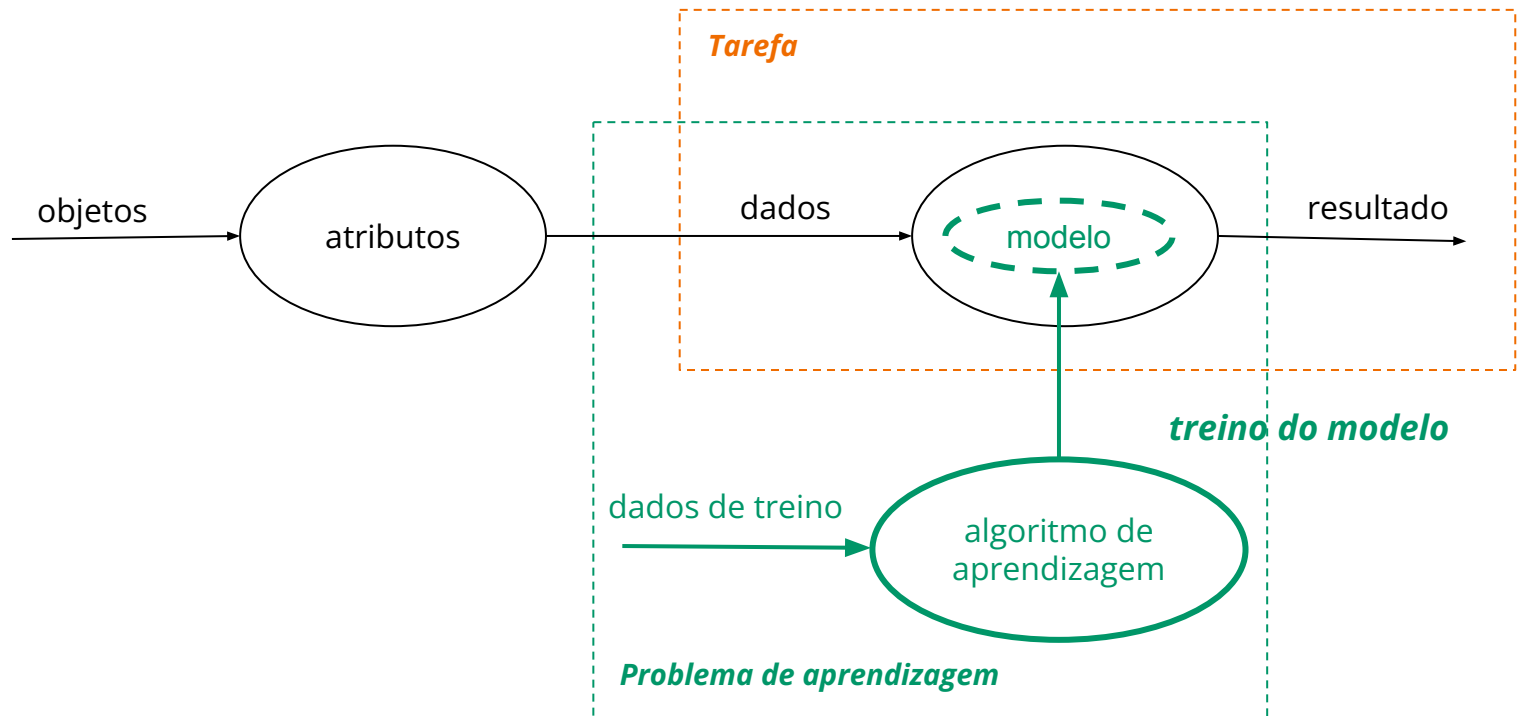
Regularização

Regressão logística

---

# Sistema de AA

# Sistema de AA



# Algoritmos de aprendizagem

- Saber como funcionam permite
  - escolher o modelo apropriado
  - selecionar o algoritmo certo para treino
  - selecionar um bom conj de hiperparâmetros
  - fazer debug de problemas
  - realizar análise de erros de forma eficiente

# Regressão linear

# Modelo linear

- Predição
  - soma pesada dos atributos de entrada, mais uma constante (termo *bias*, *intercept*)

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- $\hat{y}$  : valor previsto
- $n$  : num atributos
- $x_i$  : valor do atributo  $i$
- $\theta_j$  : parâmetro  $j$  do modelo
  - $\theta_0$  : termo bias
  - $\theta_1, \theta_2, \dots, \theta_n$  : pesos dos atributos

- Forma vetorial

$$\hat{y} = \mathbf{h}_\theta(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$$

- $\boldsymbol{\theta}$  : vetor de parâmetros
- $\mathbf{x}$  : vetor de atributos
  - $x_0 = 1$
- $\boldsymbol{\theta} \cdot \mathbf{x}$  : produto escalar
- $\mathbf{h}_\theta$  : função hipótese c/ parâmetros  $\boldsymbol{\theta}$

- Forma matricial

$$\hat{y} = \boldsymbol{\theta}^T \mathbf{x}$$

# Como escolher o modelo?

- Treinar
  - escolher os parâmetros que **melhor se ajustam** aos dados de treino
- Medida de ajuste
  - RMSE (raíz da média dos erros quadrados)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left( h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

- Treinar
  - encontrar o valor de  $\theta$  que minimiza RMSE

# Função de custo

- minimizar RMSE = a minimizar **MSE** (média dos erros quadrados)
  - MSE mais simples de calcular
  - gera o mesmo resultado

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m \left( \boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

- função de custo <> função desempenho
- Função de custo
  - usada para otimizar os parâmetros
- Função de desempenho
  - usada para avaliar o modelo

# Equação normal

- Solução analítica

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

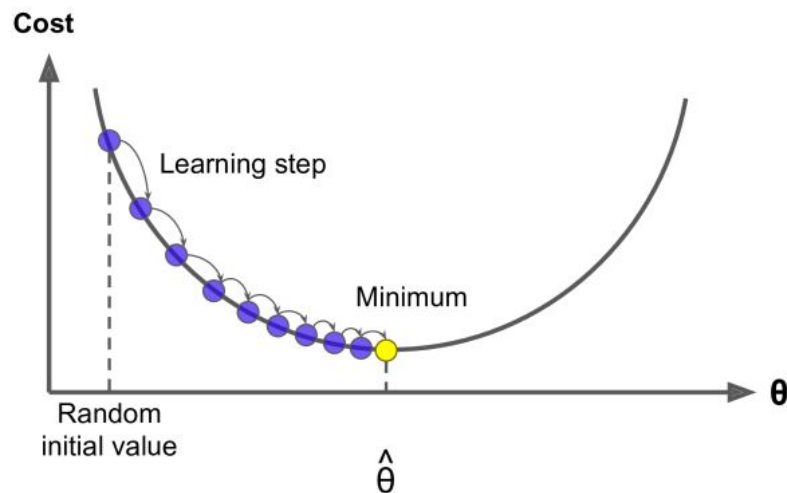
- $\hat{\boldsymbol{\theta}}$  : valor de  $\boldsymbol{\theta}$  que minimiza a função de custo
  - $\mathbf{y}$  : vetor de valores resultado ( $y^1$  a  $y^m$ )
  - Complexidade:  $O(n^{2.4})$  a  $O(n^3)$
- Aproximação SVD (*single value decomposition*)
  - Complexidade:  $O(n^2)$

# Descida do gradiente

# Descida do gradiente

- Algoritmo de otimização
  - ajustar iterativamente os parâmetros (que minimizam a função de custo)
- Modo de funcionamento
  - mede o gradiente da função de custo (tendo em conta os parâmetros)
  - progride na direção do gradiente descendente
  - mínimo: gradiente igual a zero

*Funciona melhor em problemas com muitos atributos ou muitos exemplos!*

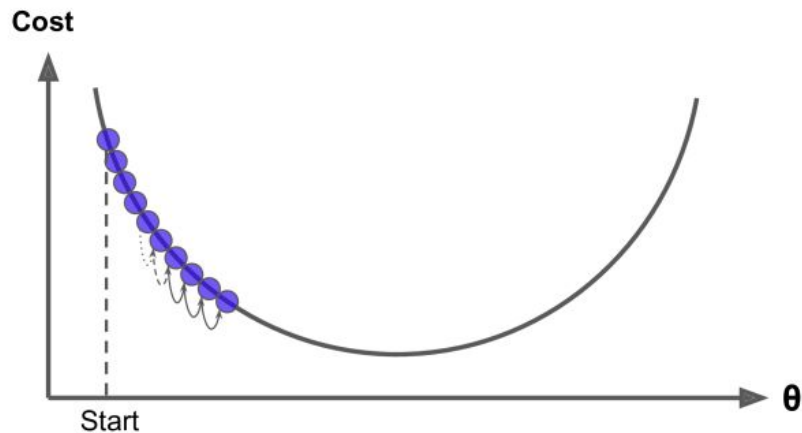


# Ritmo de aprendizagem<sup>1</sup>

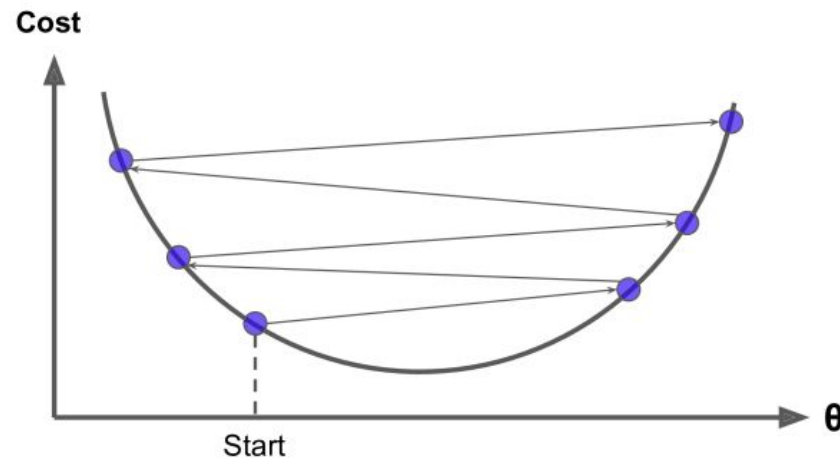
<sup>1</sup>Do inglês, *learning rate*

tamanho do passo na descida

- muito baixo: muitas iterações para convergir

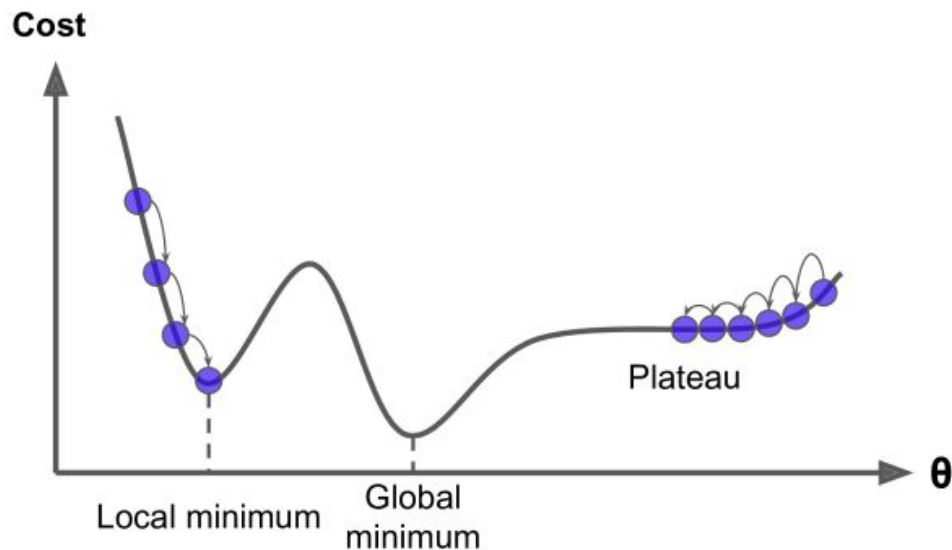


- muito alto : algoritmo pode divergir



# Funções de custo

- nem todas são regulares
  - podem existir buracos, cristas, planaltos e outros tipos de irregularidades
  - dificultam a convergência para o mínimo
- problemas
  - mínimo locais
  - demasiado tempo para convergir
  - paragem antes do tempo

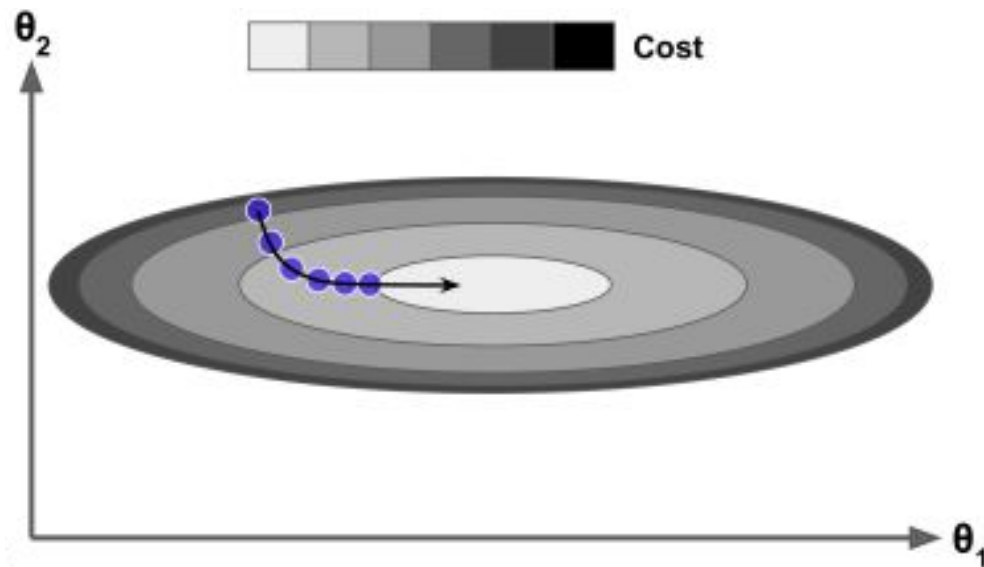
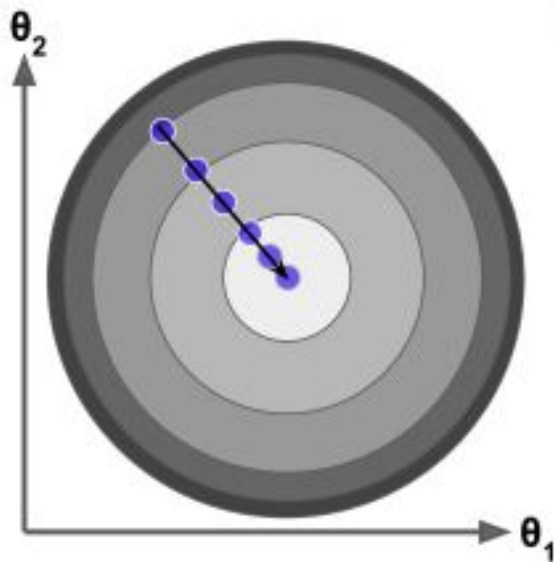


# MSE

- Características
  - função convexa
    - não existem mínimos locais
  - declive não muda repentinamente
- Garante aproximação arbitrária ao mínimo global
  - se o nº de iterações for suficiente
  - se o ritmo de aprendizagem não for muito alto

# Função de custo e valores dos atributos

- Atributos 1 e 2 com valores semelhantes
- Atributo 1 com valores muito mais pequenos (maiores variações em  $\theta_1$  para produzir a mesma alteração na função de custo)



# Descida do gradiente

- É essencial que todos os atributos tenham escalas semelhantes
  - senão demora muito mais tempo a convergir
  - Alterar a escala dos atributos
    - StandardScaler
    - MinMaxScaler
- Pesquisa no espaço dos parâmetros do modelo
  - mais parâmetros -> mais dimensões -> pesquisa mais difícil

# Batch gradient descent

- Gradiente da função de custo para cada parâmetro  $\theta_j$ 
  - derivada parcial com respeito a  $\theta_j$

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^m \left( \boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

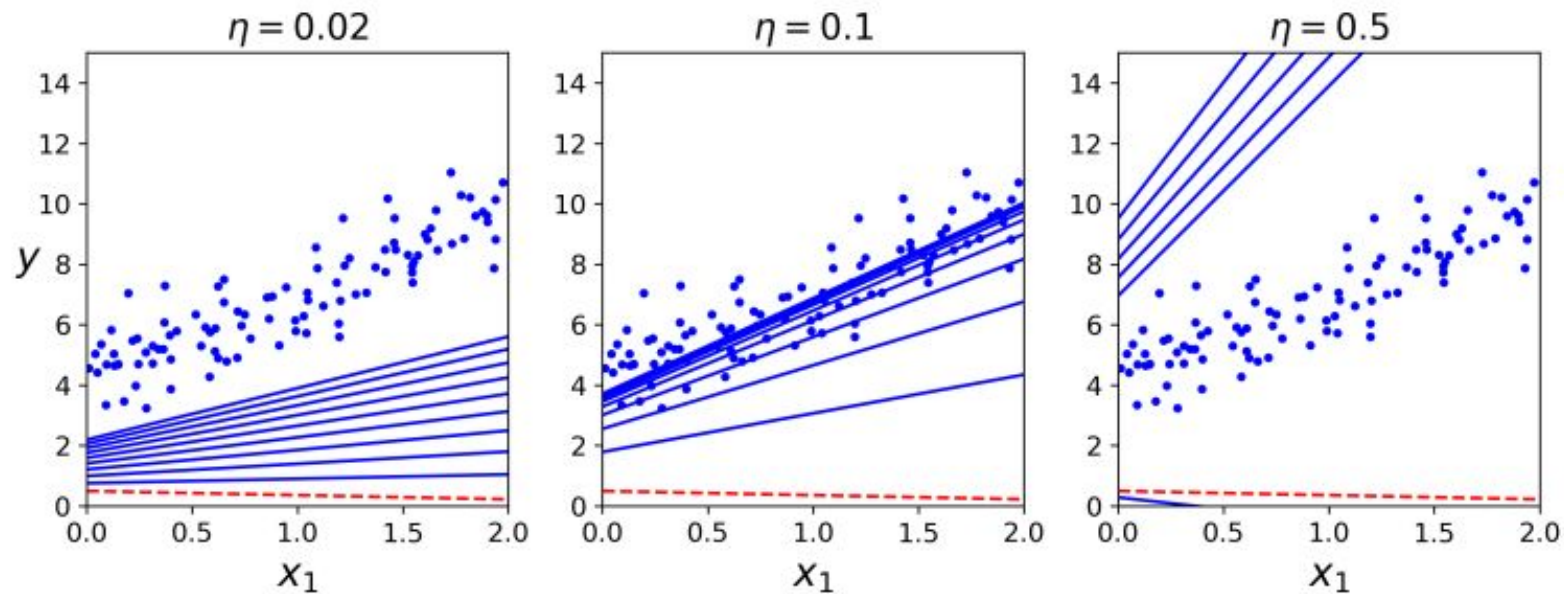
cálculo sobre **todo o conjunto de treino** em cada passo...

... **muito lento** para conj treino grandes!

# Passo da descida do gradiente

$$\theta^{(\text{next step})} = \theta - \eta \nabla_{\theta} \text{MSE}(\theta)$$

- $\eta$  : ritmo de aprendizagem

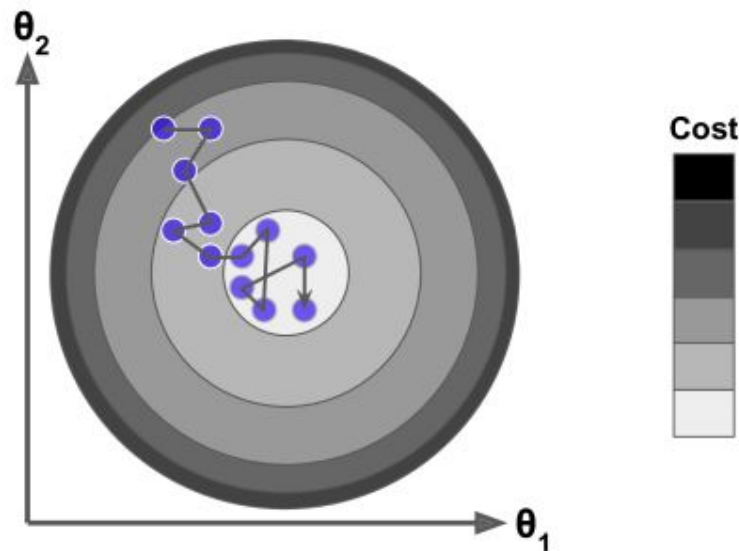


# Número de iterações

- Deve ser limitado
  - para eliminar modelos que demoram muito tempo a convergir
    - muito pequeno -> longe do ótimo no final
    - muito grande -> desperdício de tempo quando os parâmetros já não mudam
- Estratégia
  - escolher um  $n^\circ$  muito alto
  - interromper pesquisa quando gradiente for muito pequeno
    - acontece quando o algoritmo (quase) alcança o mínimo
    - norma menor que  $\varepsilon$ 
      - $\varepsilon$  : tolerância
    - $n^\circ$  iterações necessárias:  $O(1/\varepsilon)$

# Stochastic gradient descent (SGD)

- Em cada passo
  - escolhe uma instância **aleatoriamente**
  - calcula o gradiente
  - atualiza os pesos
- Características
  - iteração muito mais rápida
  - algoritmo muito menos regular
    - função de custo sobe e desce
    - apenas decresce na média
    - perto do mínimo, continua a saltar sem estabilizar
      - parâmetros finais são bons mas não ótimos



# Aleatoriedade

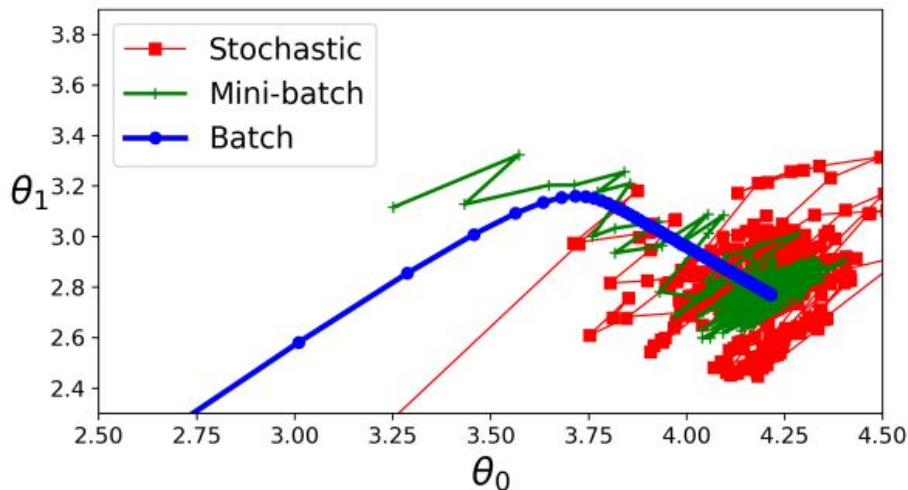
- Dilema
  - Ajuda a saltar de mínimos locais
    - bom para funções irregulares
  - Não pára no mínimo
- Solução
  - reduzir o ritmo de aprendizagem gradualmente
    - começa com passos largos para fugir de mínimos locais
    - torna-se cada vez mais pequeno para estabilizar o algoritmo
  - *learning schedule*
    - função que determina o ritmo de aprendizagem a cada iteração

# Instâncias avaliadas

- Escolha aleatória
  - instâncias escolhidas várias vezes
  - instâncias que não são escolhidas
- Alternativa (que garante a escolha de todas as instâncias)
  - em cada época
    - **shuffle** do conjunto
    - iterar sobre todas as instâncias
  - geralmente converge mais devagar!
- Época = ronda de  $m$  avaliações
  - $m$  : nº exemplos

# Mini-batch gradient descent

- Calcula o gradiente sobre um **conjunto aleatório pequeno** de instâncias
- Vantagens sobre SGD
  - boost de desempenho através da otimização de operações de matrizes em hardware (GPU)
  - progride no espaço de parâmetros de forma menos errática (c/ mini-batches grandes)
- Desvantagens
  - mais difícil escapar mínimos locais



# Comparação de algoritmos

- $m$  : n° de exemplos
- $n$  : n° atributos

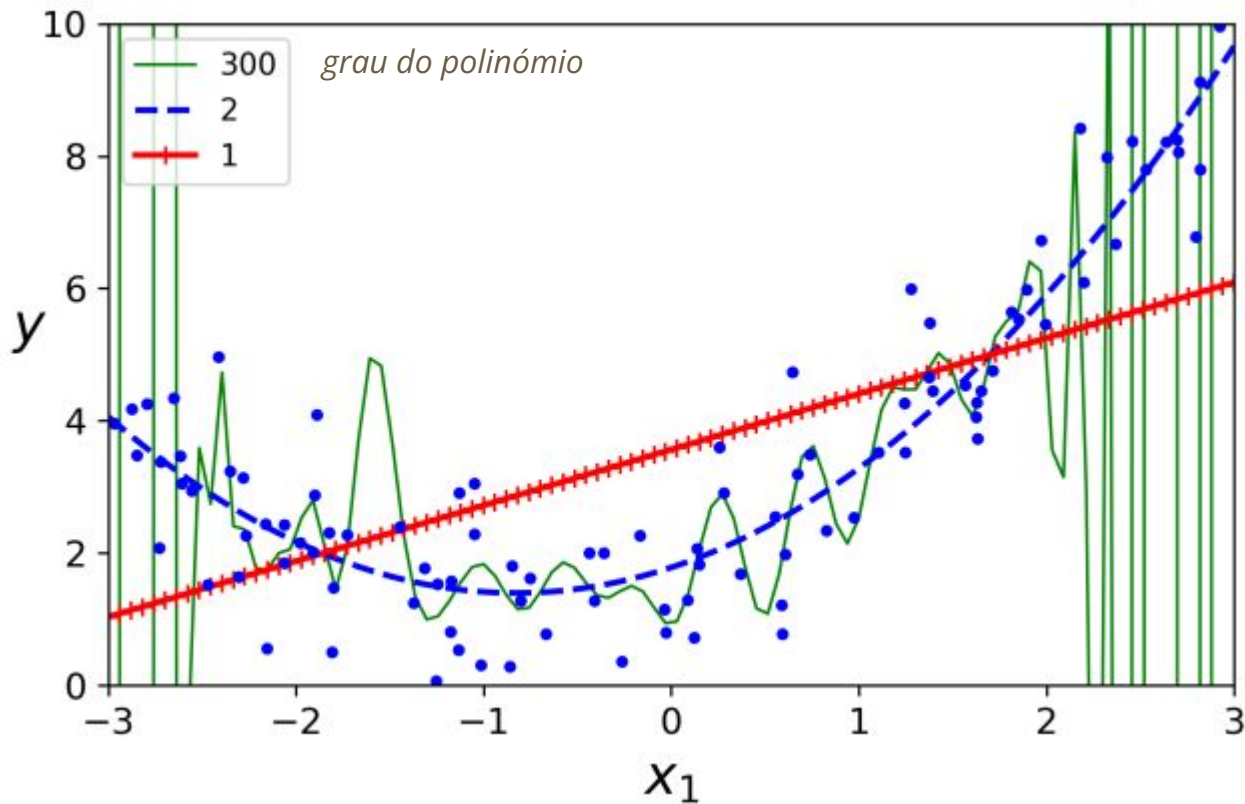
| Algorithm       | Large $m$ | Out-of-core support | Large $n$ | Hyperparams | Scaling required | Scikit-Learn     |
|-----------------|-----------|---------------------|-----------|-------------|------------------|------------------|
| Normal Equation | Fast      | No                  | Slow      | 0           | No               | n/a              |
| SVD             | Fast      | No                  | Slow      | 0           | No               | LinearRegression |
| Batch GD        | Slow      | No                  | Fast      | 2           | Yes              | SGDRegressor     |
| Stochastic GD   | Fast      | Yes                 | Fast      | $\geq 2$    | Yes              | SGDRegressor     |
| Mini-batch GD   | Fast      | Yes                 | Fast      | $\geq 2$    | Yes              | SGDRegressor     |

# Regressão polinomial

# Regressão polinomial

- Utiliza modelos lineares para ajustar dados não lineares
- Como?
  - adiciona potências dos atributos como novos atributos
  - treina um modelo linear sobre o conj estendido de atributos
- Algoritmo capaz de encontrar relações entre atributos
- Exemplo
  - atributos:  $a, b$
  - grau: 3
  - novos atributos:  $a^3, b^3, a^2, b^2, ab^2, a^2b, ab$

# Ajuste aos dados



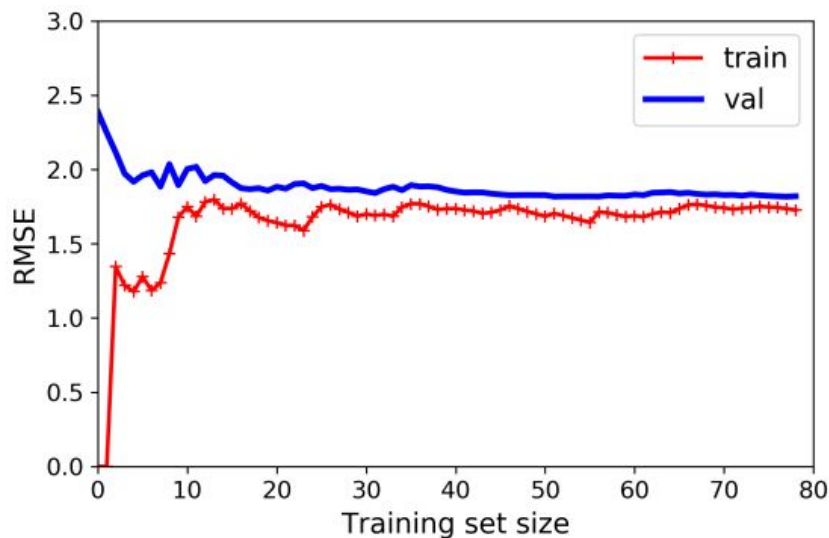
# Curvas de aprendizagem

# Poder de generalização

- Usar validação cruzada para estimar o desempenho
  - Sobre-ajustamento
    - modelo com bom desempenho nos dados de treino
    - generaliza mal de acordo com as métricas de validação cruzada
  - Sub-ajustamento
    - modelo com mau desempenho em ambos os conjuntos
- Analisar as **curvas de aprendizagem**
  - Gráfico do desempenho dos conj. de treino e validação como função tamanho do conj. treino (ou da iteração do treino)

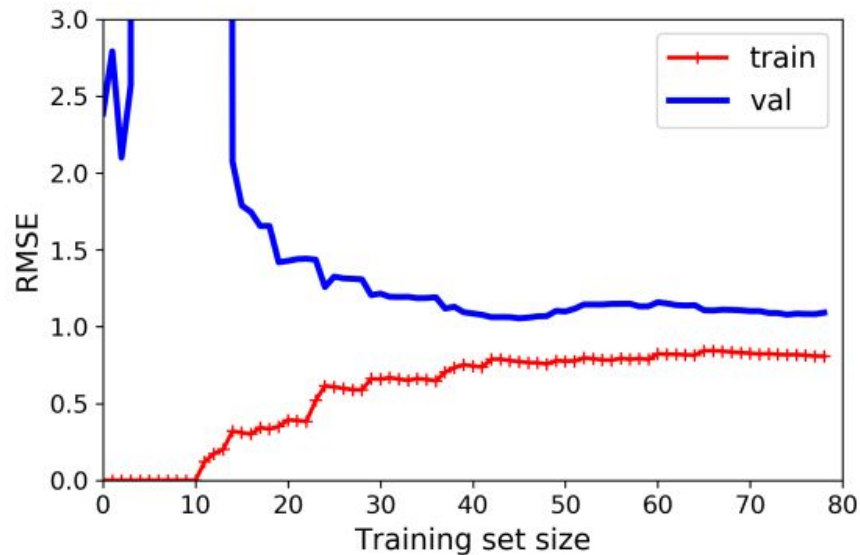
# Exemplo: regressão linear

- conj treino
  - ajuste perfeito c/ 1-2 instâncias
  - c/ + instâncias torna-se impossível
    - dados c/ ruído, problema não linear
  - atinge um planalto
    - mais instâncias não alteram o erro médio
- conj val
  - c/ poucas instâncias é incapaz de generalizar
  - depois consegue aprender
  - atinge um planalto perto da curva de treino
- Problema: sub-ajustamento
  - adicionar mais instâncias não ajuda!



# Exemplo: polinômio grau 10

- conj treino
  - erro muito menor que o do modelo linear
- conj val
  - existe um intervalo entre as 2 curvas
  - desempenho significativamente melhor no conj treino
- Problema: sobre-ajustamento
  - adicionar mais instâncias permite a aproximação das duas curvas



# Compromisso enviesamento-variância

- Erro de generalização = soma de 3 erros muito distintos
- Enviesamento (*bias*)
  - premissas erradas
  - modelo com enviesamento alto (assumir linear quando é quadrático) -> sub-ajustamento
- Variância
  - sensibilidade a variações pequenas nos dados de treino
  - modelo de alta variância alta (muitos graus de liberdade) -> sobre-ajustamento
- Erro irreduzível
  - devido ao ruído nos dados
  - a única forma de reduzir é limpar os dados

# Regularização

# Redução do sobre-ajustamento

- Regularizar o modelo = menos graus de liberdade
  - modelo polinomial
    - reduzir o grau do polinómio
  - modelo linear
    - restringir os pesos do modelo

# Regressão Ridge

- Regularização de Tikhonov
- Adiciona um termo de regularização à função de custo
  - modelo ajusta-se aos dados
  - mantém os pesos o menor possível
- Parâmetros
  - $\alpha$  : controla quanto se quer regularizar
    - $\alpha=0$  -> regressão linear
    - $\alpha$  alto -> pesos próximos de zero -> linha horizontal no valor médio dos dados
- Função de custo

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2$$

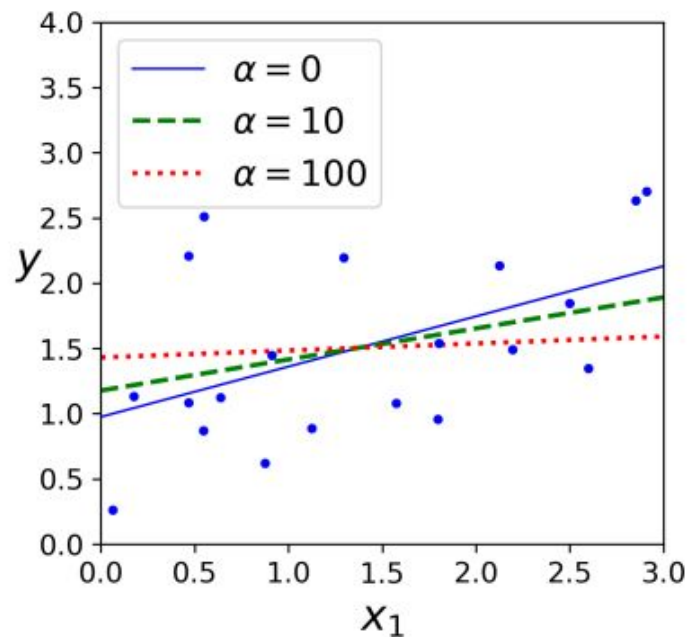
$\theta_0$  não é regularizado

# Funções de custo e desempenho

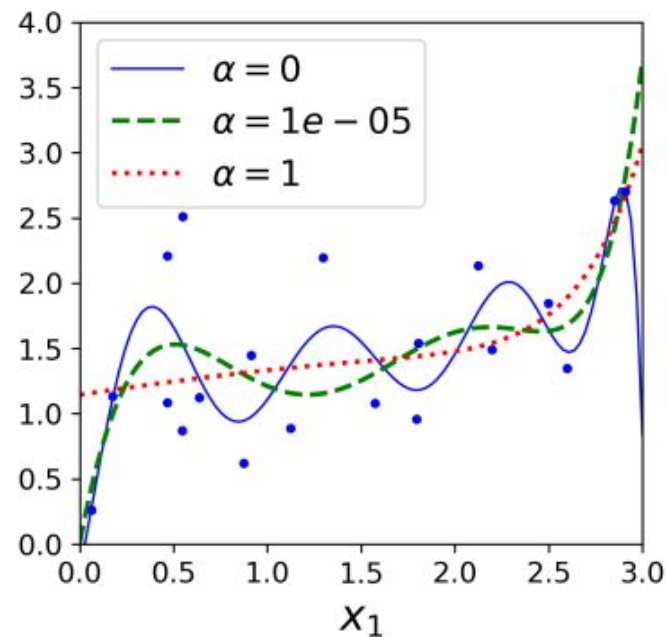
- Função de custo
  - guia a pesquisa do modelo (otimização dos pesos)
- Função de desempenho
  - medida para avaliar o desempenho do modelo
- Podem ser diferentes
  - a regularização é apenas aplicada no treino
  - a função de custo deve ter derivadas faceis de otimizar
  - a função de desempenho deve ser próxima do objetivo final

# Exemplo: dados lineares

Modelo Ridge



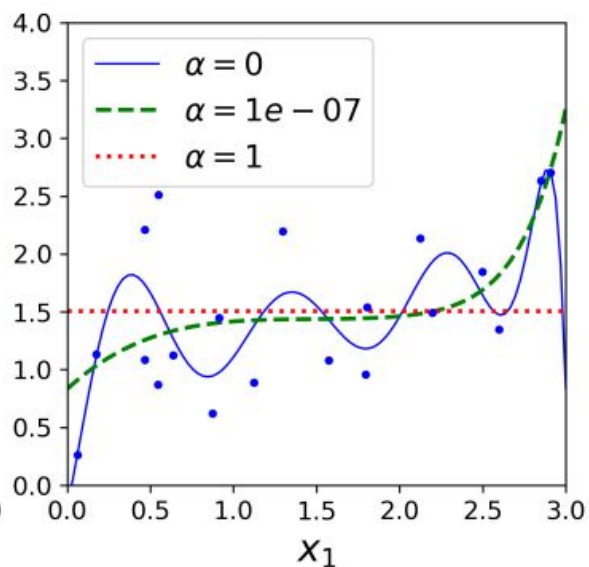
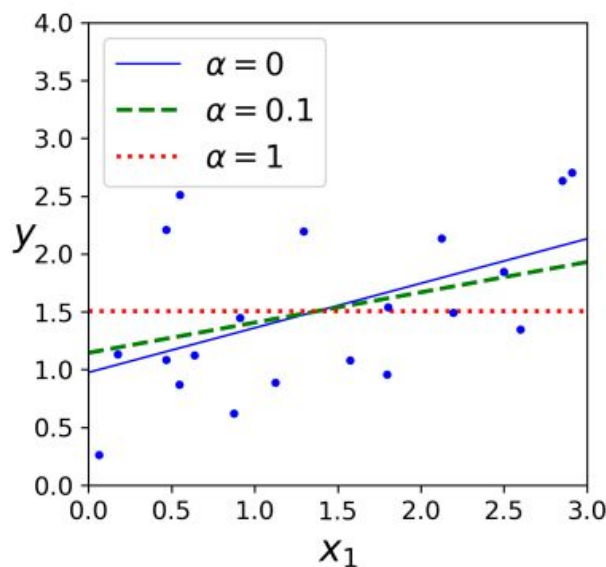
Modelo Ridge com dados escalados sobre atributos polinomiais (grau 10)



# Regressão Lasso

- *Least Absolute Shrinkage and Selection Operator Regression*
- Função de custo

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \sum_{i=1}^n |\theta_i|$$



# Termos de regularização

- Regressão Ridge
  - metade do quadrado da norma  $\ell_2$  do vetor de pesos

$$\frac{1}{2} \sum_{i=1}^n \theta_i^2 \quad \frac{1}{2} (\|\mathbf{w}\|_2)^2$$

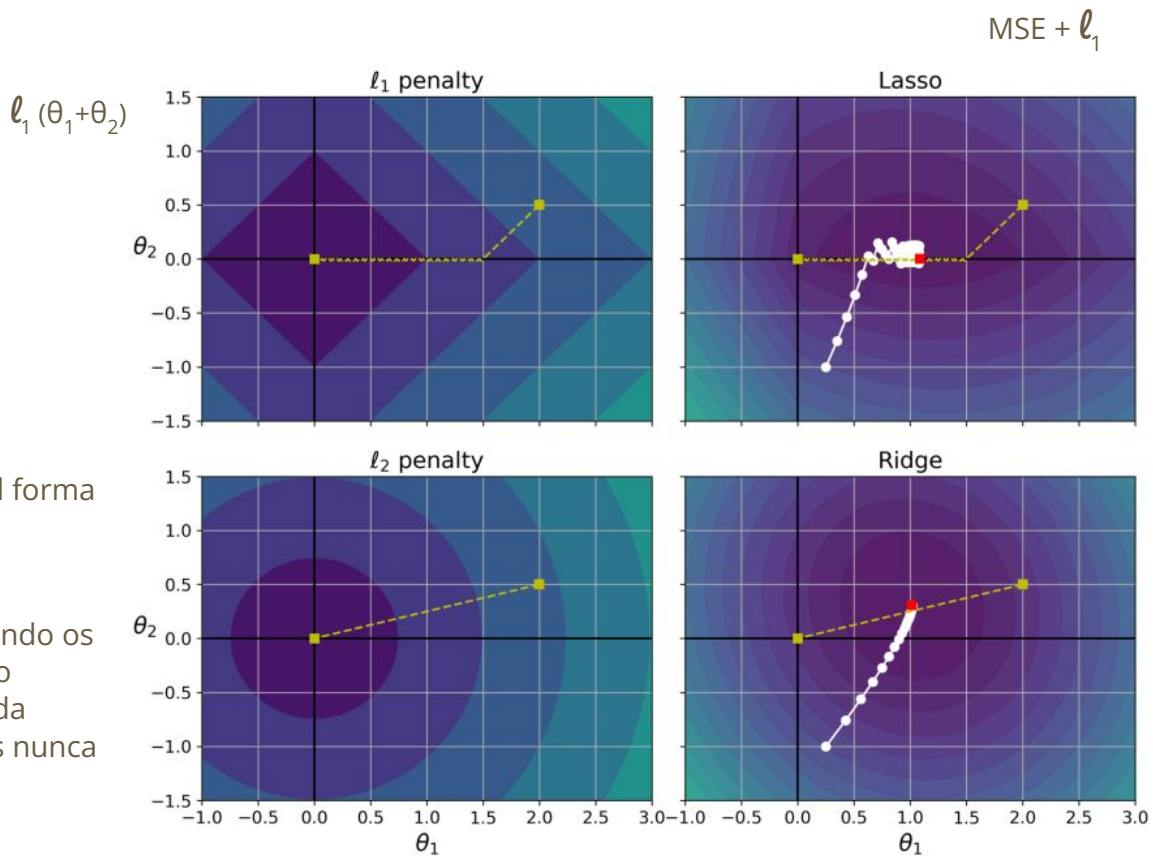
- Regressão Lasso
  - norma  $\ell_1$  do vetor de pesos

$$\sum_{i=1}^n |\theta_i| \quad \|\mathbf{w}\|_1$$

- tende a **eliminar os pesos** dos atributos menos importantes
  - seleção automática de atributos
  - geração de um modelo esparsos

# Regularização Lasso e Ridge

- Perda
  - $\ell_1, \ell_2$
  - parâmetros iniciais
    - $\theta_1=2, \theta_2=0.5$
- Gradient descent
  - Lasso, Ridge
  - parâmetros iniciais
    - $\theta_1=0.25, \theta_2=-1$
- Lasso
  - decrementa os parâmetros de igual forma
  - resalta à volta do ótimo global
- Ridge
  - gradientes tornam-se menores quando os parâmetros se aproximam do ótimo
  - parâmetros ótimos aproximam-se da origem com o incremento do  $\alpha$  mas nunca são eliminados



# Elastic Net

- Combina regressão Ridge e Lasso
  - o termo de regularização é uma mistura de ambos
  - controlável através de parâmetro  $r$ 
    - $r=0$  -> regressão ridge
    - $r=1$  -> regressão Lasso

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r\alpha \sum_{i=1}^n |\theta_i| + \frac{1-r}{2}\alpha \sum_{i=1}^n \theta_i^2$$

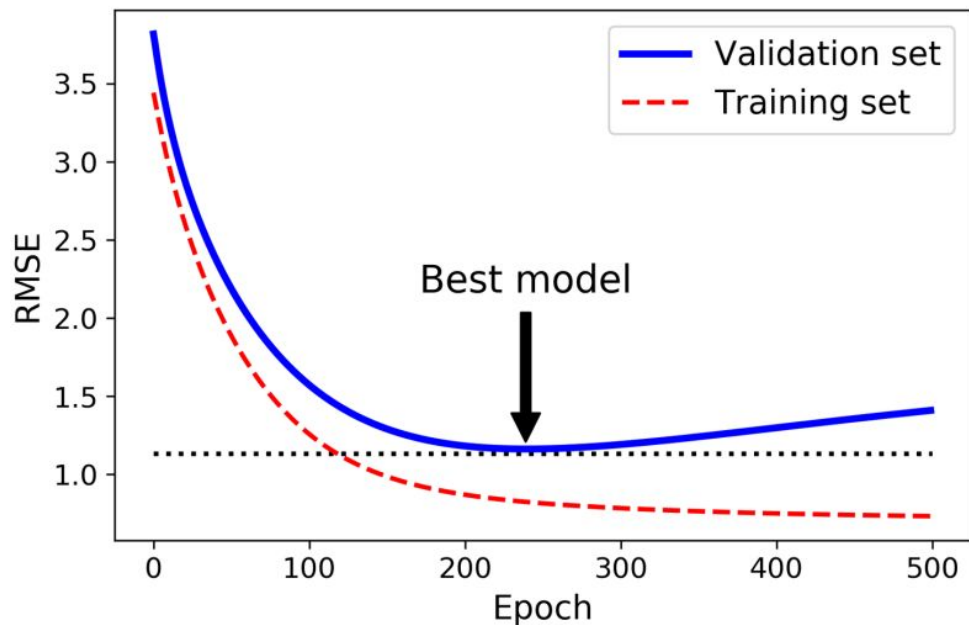
# Como escolher?

- Usar sempre alguma regularização
  - evitar regressão linear
- Por omissão
  - Ridge
- Com suspeita que apenas alguns atributos são úteis
  - Elastic Net
  - Lasso
    - pode ter comportamento errático quando
      - $n^\circ$  atributos é maior que  $n^\circ$  exemplos
      - vários atributos têm correlação forte

# Terminação precoce<sup>1</sup>

<sup>1</sup>Do inglês, *Early stopping*

- Outra forma de regularizar algoritmos iterativos
- Funcionamento
  - parar o treino assim que o erro de validação atinge um mínimo
- *"beautiful free lunch"*
  - Geoffrey Hinton



# Terminação precoce e descida do gradiente

- Mini-batch e stochastic GD
  - as curvas não são tão suaves
  - pode ser difícil saber quando se atingiu o mínimo
- Solução
  - parar quando estiver confiante que o modelo não será melhor
  - escolher os parâmetros do ponto onde o erro de validação é mínimo

# Regressão logística

# Regressão logística

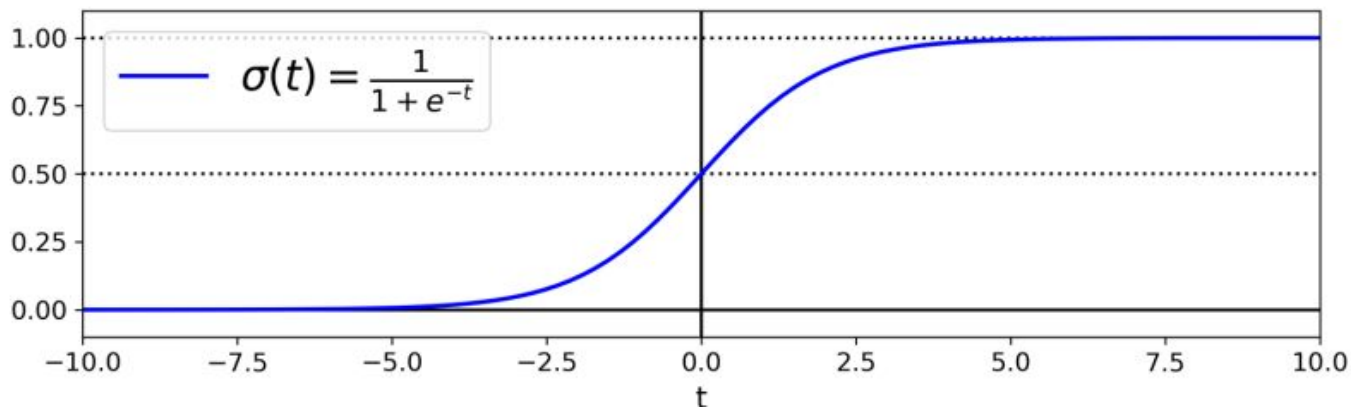
- Estima a probabilidade de uma instância pertencer a uma classe
  - probabilidade > 50% -> instância pertence a essa classe
  - pode ser usada como classificador binário
- Como funciona?
  - calcula uma soma pesada dos atributos de entrada (mais o termo bias)
  - aplica a função logística ao resultado (em substituição do resultado em si)
    - $\sigma()$
- Estimação de probabilidade

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\mathbf{x}^T \boldsymbol{\theta})$$

# Função logística

- função sigmoide  $\sigma$ 
  - tem como resultado um valor entre 0 e 1
- valor *logit*
  - $t$

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$



# Previsão

- Estimação da probabilidade de pertencer à classe positiva

$$\hat{p} = h_{\theta}(\mathbf{x})$$

- Previsão

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

# Treino e função de custo

- Objetivo
  - um modelo que estima probabilidades
    - altas para instâncias positivas
    - baixas para instâncias negativas

- Função de custo

$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1 - \hat{p}) & \text{if } y = 0 \end{cases}$$

- Comportamento
  - valor alto quando probabilidade estimada
    - é perto de 0 para instâncias positivas
    - é perto de 1 para instâncias negativas
  - valor próximo de 0 quando probabilidade estimada
    - é perto de 1 para instâncias positivas
    - é perto de 0 para instâncias negativas

# log loss

- Função de custo sobre o todo o conj de treino

- média sobre todas as instâncias de treino

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \left[ y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

- Função convexa

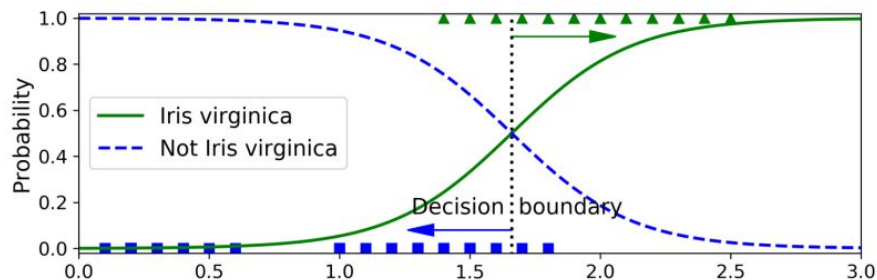
- descida do gradiente garante encontrar o mínimo global
- se o ritmo de aprendizagem não for demasiado grande e forem executadas iterações suficientes

- Derivadas parciais

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \left( \sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

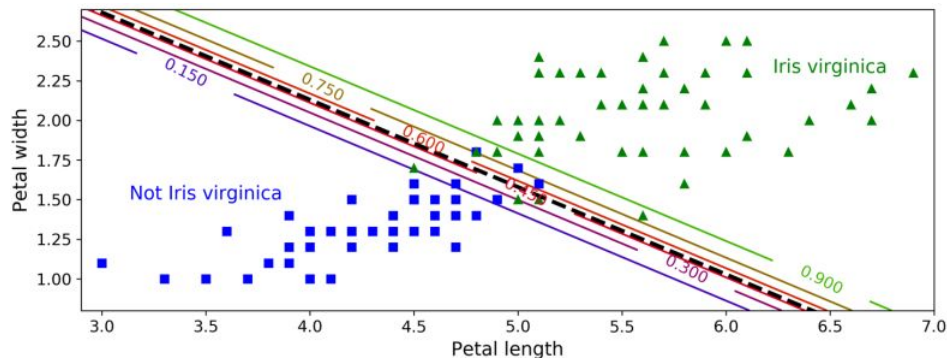
# Fronteiras de decisão

- Problema
  - detetar "Iris Virginica" a partir do atributo "petal width"



- Problema
  - detetar "Iris Virginica" a partir dos atributos "petal length" e "petal width"

diferentes valores de  $\alpha$  = diferentes fronteiras



# Regularização

- Penalizações
  - $\ell_1$
  - $\ell_2$
- Parâmetro de regularização
  - C
    - C=complexidade do modelo
    - inverso de  $\alpha$
    - maior o C -> menor regularização

# Regressão softmax

- *Multinomial logistic regression*
- Generalização para suportar múltiplas classes diretamente
- Funcionamento
  - calcula a pontuação  $s_k(\mathbf{x})$  para cada classe  $k$
  - estima probabilidade de cada classe através da função softmax (normalized exponential)
- Pontuação (score)
  - logit ou log-odd

$$s_k(\mathbf{x}) = \mathbf{x}^\top \boldsymbol{\theta}^{(k)}$$

# Previsão

- Função softmax
  - $K$  : n° de classes
  - $\mathbf{s}(\mathbf{x})$  : scores de cada classe para a instância  $\mathbf{x}$
  - $\sigma(\mathbf{s}(\mathbf{x}))_k$  : estimacão de probabilidade da instância  $\mathbf{x}$  pertencer à classe  $k$ , dados os scores de cada classe

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

- Previsão
  - classe com maior probabilidade
  - classe com maior score

$$\begin{aligned}\hat{y} &= \operatorname{argmax}_k \sigma(\mathbf{s}(\mathbf{x}))_k = \operatorname{argmax}_k s_k(\mathbf{x}) \\ &= \operatorname{argmax}_k \left( \left( \boldsymbol{\theta}^{(k)} \right)^\top \mathbf{x} \right)\end{aligned}$$

# Função de custo

- Objetivo
  - modelo que estima uma probabilidade alta para a classe correta
  - minimizar cross-entropy
    - penaliza modelos que estimam probabilidades baixas para a classe correta
- Cross-entropy
  - $y_k^{(i)}$ : probabilidade da instância  $i$  pertencer à classe  $k$ 
    - 1 se instância pertence à classe  $k$
    - 0 se instância pertence a outra classe

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

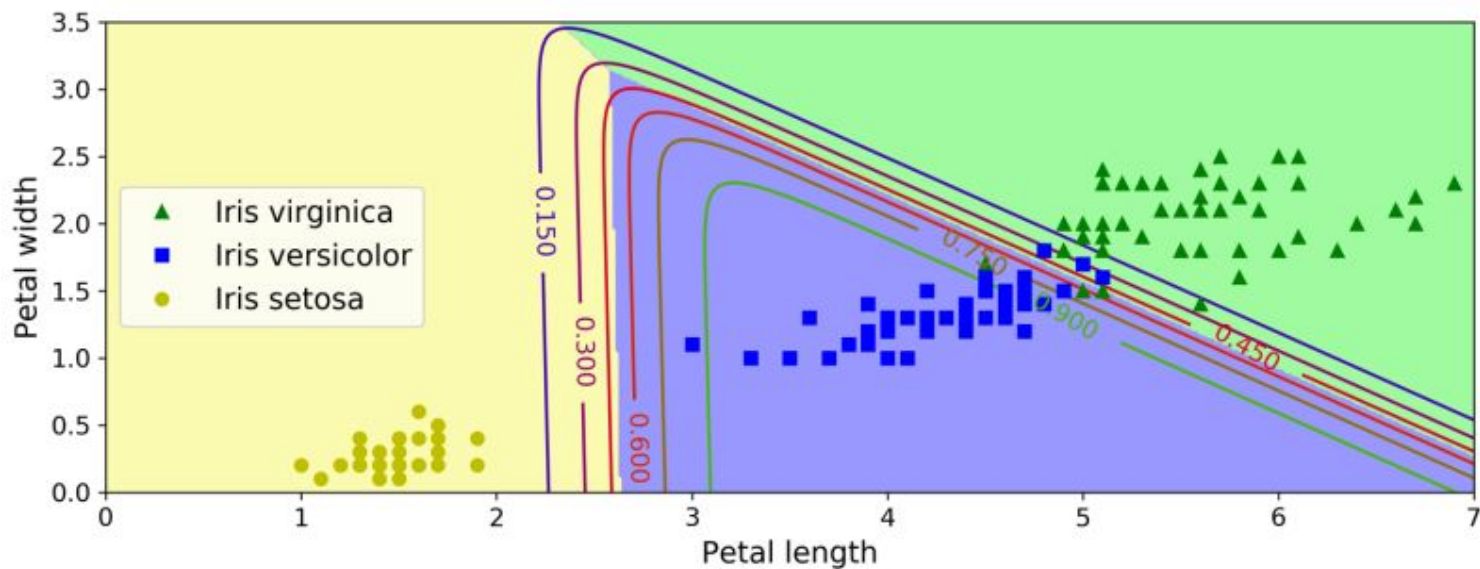
- Derivadas parciais

$$\nabla_{\theta^{(k)}} J(\Theta) = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}$$

# Fronteiras de decisão

- Problema
  - Classificar flores iris em uma das 3 classes

diferentes valores de  $C$  = diferentes fronteiras



# Bibliografia

## Capítulo 4

*Hands-On Machine Learning with  
Scikit-Learn, Keras & TensorFlow*

---